

System Architecture Specification for DeviceCheck: A Local-First Windows Hardware Forensic and Identity Verification Engine

The precise identification of physical hardware on Windows environments is frequently obscured by the operating system’s reliance on superficial, driver-supplied display strings. Standard administrative interfaces, such as the Windows Device Manager, parse active driver installation manifests to present hardware names that are often generic, incomplete, or tailored strictly to vendor marketing preferences.¹ This architectural limitation introduces substantial difficulty when executing system audits, verifying hardware authenticity, or debugging low-level peripheral issues.

To resolve these challenges, DeviceCheck is established as a high-fidelity, local-first hardware identity engine running on PowerShell 7. By decoupling hardware identification from the active driver layer, DeviceCheck constructs an independent, multi-layered synthesis model that queries physical bus interfaces, decodes local driver storage catalogs, evaluates open-source hardware registries, and correlates platform system topology. This technical specification defines the schemas, ingestion pipelines, scoring algorithms, and validation frameworks required to construct and deploy the DeviceCheck identity and verification engine.

Hardware Identity Source Registry

To perform deterministic hardware resolution in air-gapped or restricted operational environments, DeviceCheck integrates a diverse registry of local databases, system configurations, and auxiliary external metadata caches.

Source Name	ID Types Resolved	Primary URL / Repository	Licenses & Redistribution	Update Frequency	Format	Integration Strategy	Trust Level	Key Fields to Normalize
PCI ID Repository ³	PCI Vendors, Devices, Subsystems,	https://pci-ids.ucw.cz/ ³	GPL v2+ or 3-Clause BSD ³	Daily automated snapshots ³	Flat Indexed Text File ³	Pre-compiled JSON cache ³	High ³	VendorId, DeviceId, SubsystemVendorId

	Classes, Subclasses ³							d, SubsystemDeviceId, ClassName ³
USB ID Repository ⁴	USB Vendors, Products, Device Classes, Subclasses, Protocols ⁴	http://www.linux-usb.org/usb-id/s.html ⁴	GPL v2+ or 3-Clause BSD ⁴	Daily automated snapshots ⁴	Flat Indented Text File ⁴	Pre-compiled JSON cache ⁴	High ⁴	VendorId, ProductId, DeviceClass, DeviceSubClass, Protocol ⁴
PNP & ACPI Registry ⁵	ACPI and Plug-and-Play Vendor Codes ⁵	https://uefi.org/PP_ACP/Registry ⁶	Proprietary (UEFI Forum), redistributable via hwdata ⁵	Periodically updated ⁵	HTML / CSV ⁵	Build-time JSON generation ⁸	High ⁵	PnpVendorId, VendorName, ApprovalDate ⁹
Linux hwdb ¹⁰	PCI, USB, SDIO, Bluetooth, Autosuspend properties	https://github.com/systemd/systemd/tree/master/hwd	LGPL v2.1+	Continuous (with systemd releases) ¹²	Key-Value Rules ¹⁰	Parsed & indexed JSON	Medium-High ¹³	ModaliasPattern, VendorName, ModelName, Autos

	rties ¹⁰	b.d ¹⁰						uspen d ¹⁰
ALSA Use Case Mana ger ¹⁴	USB Audio profile s, endpo ints, mixer layout s ¹⁴	https:// github.com /alsa-p roject/ alsa-u cm-co nf ¹⁴	BSD 3-Clau se	Contin uous ¹²	Text Config (ALSA parser syntax) ¹⁶	Indexe d JSON lookup	High ¹⁷	UsbVe ndorPr oduct Regex, Codec Model, JackM appin g, Profile Name ¹⁶
Wind ows Driver Store	Hardw are IDs, Comp atible IDs, Driver INF paths, Servic es	Local Syste m (C:\Wi ndows \Syste m32\D riverSt ore\Fil eRepo sitory)	OEM / Micros oft Propri etary	Syste m-de pende nt (On driver install)	INF (INI-lik e format)	Direct parsin g of local active INFs	High	Hardw areId, Provid er, Driver Versio n, Sectio nNam e, Servic e
Micro soft Updat e Catal og	Driver CAB manife sts, hardw are target s, INF metad ata	https:// www. catalo g.upd ate.mi crosof t.com	Micros oft Propri etary	Contin uous	CAB / XML	On-de mand API query	High	Hardw areId, Updat eId, Driver Provid er, Suppo rtedO S
EDID	Monit	Decod	Public	Static	Binary	Local	High ²³	Eisald,

Registries ¹⁹	or manufacture, product codes, timings ¹⁹	ed from Registry / UEFI PNP databases ¹⁹	Domain / UEFI Registry ¹⁹	standard ²¹	payload (128/256 bytes) ²¹	runtime parser		ProductCode, WeekYearOfManufacture, PreferredTiming ¹⁹
TechPower Up GPU DB	GPU Board models, clock speeds, VRAM, ASIC designs	https://www.techpowerup.com/gpu-specs/	Proprietary (API rate-limited)	Continuous	HTML / JSON API	On-demand API / SQLite mirror	High	PciVendorId, PciDeviceId, SubsystemId, GpuName, MemoryType
Snappy Driver Installer	Offline driver pack indexes, hardware-to-INF mappings	https://sdi-tool.org/	GPL v3	Periodically updated	XML / INI	Indexed offline database	Medium	HardwareId, DriverName, CabinetFile, Rank
LVFS / fwupd	Firmware targets, BIOS	https://fwupd.org/downloads/	LGPL v2.1+	Continuous	XML / GZ	Cached JSON catalog	High	Guid, PciUsbId, FirmwareVer

	versions, device security states							sion, Vendor, Release Date
OEM Specifications	Baseboard configurations, integrated controllers	Local curations from manufacturer support sheets	Public Domain metadata	Motherboard- dependent	JSON / Map structures	Bundled model matrices	High	Motherboard Model, Audio Codec, LanController, WlanController

The primary local repositories, `pci.ids` and `usb.ids`, are structured as hierarchical, tab-indented flat files where vendor definitions exist at the root level.³ Under each vendor block, device entries are nested with a single tab, and subsystem or subvendor configurations are nested with two tabs.³

The Plug-and-Play (PNP) registry provides resolution for legacy, ACPI, and motherboard-integrated system devices.⁵ While the UEFI Forum hosts the authoritative PNP and ACPI vendor registries, the flat database `pnplib` curated within the open-source `hwdata` repository consolidates these vendor mappings.⁵ It resolves the 3-letter alphanumeric PNP codes (e.g., `PNP0A03` for PCI Bus, `PNP0C02` for Motherboard Resources, and vendor codes like `AUS` for ASUSTek or `MSI` for Micro-Star International).⁷

The integration strategy for DeviceCheck relies on pre-compiling `pci.ids`, `usb.ids`, and `pnplib` into highly compressed, key-indexed JSON objects during build time. This minimizes runtime parsing overhead on the target Windows system.

Multi-Layered Hardware Evidence Model

DeviceCheck rejects the design paradigm of collapsing hardware metadata into a single string. When a tool presents a single name (such as "Realtek Audio"), it obscures the deterministic basis of that identity. It hides whether the name came from a generic driver fallback, a vendor-provided string, or an exact silicon lookup. To solve this, DeviceCheck uses an eight-layer evidence model:



1. **Layer 1: Raw Local ID Match**

The foundational silicon identity extracted directly from the system bus (PCI, USB, ACPI). This layer is immutable and consists of the physical identifiers programmed into the component's registers (e.g., Vendor ID, Product/Device ID, and Revision Number).³

2. **Layer 2: Bus, Class, and Service Match**

Resolves the operational class, subclass, and protocol codes defined by standard specification bodies (PCI-SIG, USB-IF).³ For instance, a USB class code of 01 designates an Audio device, while a service mapping to usbaudio2.sys indicates compliance with the USB Audio 2.0 specification.

3. **Layer 3: Installed Driver and Local INF Match**

Extracted from the active driver configuration stored within the Windows Driver Store.¹ This layer parses the driver's INF manifest to extract the provider, the installation section, and the specific localized display string assigned to the hardware ID.¹

4. **Layer 4: Subsystem and Subvendor Match**

Resolves the subsystem identifiers nested within PCI and USB descriptors.³ This layer differentiates a reference silicon design from its physical OEM implementation (for example, distinguishing a Realtek chip implemented on an MSI motherboard from one on a Gigabyte motherboard).¹

5. **Layer 5: System Topology Context (SMBIOS)**

Telemetry extracted from the host's System Management BIOS (SMBIOS) tables. It maps variables such as Motherboard Manufacturer, Baseboard Product, System SKU, and

Chassis Type.¹ This layer contextualizes the environment in which the component operates.

6. **Layer 6: Open-Source Profile Match**

Correlates the hardware IDs with curated external configuration registries.¹⁵ This includes scanning Linux hwdb rules for power management quirks, and matching ALSA Use Case Manager (UCM) profile configurations (alsa-ucm-conf) to resolve complex USB audio routing topologies that standard USB descriptors do not expose.¹⁰

7. **Layer 7: Official OEM Specification Match**

An auxiliary layer that matches the combined output of Layer 1 and Layer 5 with OEM technical manuals and specification sheets. If SMBIOS indicates "MSI MAG X870 TOMAHAWK WIFI" and the raw ID indicates an unlisted MSI audio vendor endpoint (0DB0:CD0E), this layer correlates the system's official specification ("Realtek ALC4080 Codec") to infer the exact chip model.¹

8. **Layer 8: Cryptographically Attested User Evidence**

A local override layer containing operator-submitted confirmations. If a system engineer physically inspects a motherboard and confirms the presence of an exact silicon component, this attestation is cryptographically hashed, timestamped, and stored locally to serve as the absolute identity ceiling.

Normalized JSON Schema Suite

To ensure interoperability, predictability, and ease of validation within PowerShell 7, DeviceCheck defines strict JSON schemas for database storage, manifests, matching results, and test suites.

Hardware ID Database Schema (hardware-database.schema.json)

```
JSON
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "HardwareIdDatabase",
  "type": "object",
  "required": ["Metadata", "Vendors"],
  "properties": {
    "Metadata": {
      "type": "object",
      "required": [],
      "properties": {
        "BusType": {
          "type": "string",
          "enum": []
        }
      }
    }
  }
}
```

```

    "Version" "type" "string"
    "ReleaseDate" "type" "string" "format" "date"
    "GeneratorSource" "type" "string"
  }
}

"Vendors"
  "type" "object"
  "additionalProperties"
    "type" "object"
    "required"
    "properties"
      "Name" "type" "string"
      "Devices"
        "type" "object"
        "additionalProperties"
          "type" "object"
          "required" "Name"
          "properties"
            "Name" "type" "string"
            "Subsystems"
              "type" "object"
              "additionalProperties" "type" "string"

```

Source Manifest and Provenance Schema (source-manifest.schema.json)

JSON


```

|
{"$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "SourceManifest",
  "type": "object",
  "required": [
    "properties"
  ],
  "properties": {
    "SourceId": {
      "type": "string"
    },
    "SourceName": {
      "type": "string"
    },
    "SourceUri": {
      "type": "string",
      "format": "uri"
    },
    "License": {
      "type": "object",
      "required": [
        "properties"
      ],
      "properties": {
        "Type": {
          "type": "string"
        },
        "TextUri": {
          "type": "string",
          "format": "uri"
        }
      }
    },
    "Provenance": {
      "type": "object",
      "required": [
        "properties"
      ],
      "properties": {
        "FetchDateTime": {
          "type": "string",
          "format": "date-time"
        },
        "CommitHash": {
          "type": "string"
        },
        "FileSignatureSha256": {
          "type": "string",
          "pattern": "^[a-fA-F0-9]{64}$"
        }
      }
    }
  }
}

```

Match Result & Evidence Bundle Schema (match-result.schema.json)

```

JSON
|
{"$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "DeviceEvidenceBundle",
  "type": "object",
  "required": [

```

```
"properties"
  "DeviceId" "type" "string"
  "BusContext"
  "type" "object"
  "required"
  "properties"
    "BusType" "type" "string" "enum"
    "VendorId" "type" "string"
    "ProductId" "type" "string"
    "SubvendorId" "type" "string"
    "SubdeviceId" "type" "string"
    "Revision" "type" "string"
    "InterfaceId" "type" "string"
    "Class" "type" "string"
    "Subclass" "type" "string"
    "Protocol" "type" "string"
  ]
]
"Layers"
  "type" "object"
  "required"
  "properties"
    "Layer1_Raw"
    "type" "object"
    "required"
    "properties"
      "Matched" "type" "boolean"
      "VendorName" "type" "string"
      "ProductName" "type" "string"
      "EvidenceText" "type" "string"
      "Provenance" "type" "string"
    ]
  ]
  "Layer2_Class"
  "type" "object"
  "properties"
    "ClassCode" "type" "string"
    "ClassName" "type" "string"
    "Service" "type" "string"
  ]
]
  "Layer3_DriverStore"
  "type" "object"
```

```
"required" "Matched"
"properties"
  "Matched" "type" "boolean"
  "Provider" "type" "string"
  "InfFile" "type" "string"
  "Section" "type" "string"
  "DriverName" "type" "string"
}
}
"Layer5_SMBIOS"
  "type" "object"
  "properties"
    "BoardManufacturer" "type" "string"
    "BoardModel" "type" "string"
    "SystemSku" "type" "string"
  }
}
"Layer6_OpenSourceProfile"
  "type" "object"
  "properties"
    "Matched" "type" "boolean"
    "Source" "type" "string" "enum"
    "ProfileName" "type" "string"
    "InferredChipModel" "type" "string"
  }
}
"Layer7_OemSpecification"
  "type" "object"
  "properties"
    "Matched" "type" "boolean"
    "MarketingModel" "type" "string"
    "VerifiedSpecs" "type" "array" "items" "type" "string"
  }
}
}
}
"Synthesis"
  "type" "object"
  "required"
  "properties"
    "ResolvedModelName" "type" "string"
    "DerivedConfidenceScore" "type" "number" "minimum" 0 "maximum" 100
    "ExplanationSummary" "type" "string"
```



Verification and Regression Testing Schema (test-cases.schema.json)

JSON

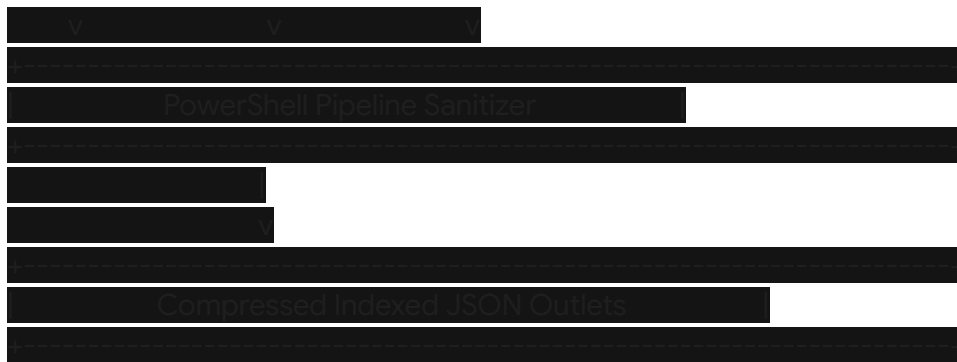
```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "HardwareRegressionTests",
  "type": "object",
  "required": [
    "properties",
    "TestSuites"
  ],
  "properties": {
    "TestSuites": {
      "type": "array",
      "items": {
        "type": "object",
        "required": [
          "SuiteName",
          "Cases"
        ],
        "properties": {
          "SuiteName": {
            "type": "string"
          },
          "Cases": {
            "type": "array",
            "items": {
              "type": "object",
              "required": [
                "TestCaseld",
                "InputHardwareId",
                "MockContext"
              ],
              "properties": {
                "TestCaseld": {
                  "type": "string"
                },
                "InputHardwareId": {
                  "type": "string"
                },
                "MockContext": {
                  "type": "object",
                  "required": [
                    "Registry",
                    "Smbios",
                    "LocalInfFiles",
                    "AlsaUcm"
                  ]
                }
              }
            }
          }
        }
      }
    }
  }
}
```

```
[REDACTED]
[REDACTED]
  "ExpectedAssertions"
    "type" "object"
    "required"
    "properties"
      "ConfidenceScoreFloor" "type" "number"
      "ExpectedFields" "type" "object"
      "ForbiddenNames" "type" "array" "items" "type" "string"
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
```

High-Performance Importer and Parser Designs

To populate DeviceCheck's offline datastores, a structured PowerShell 7 importing architecture is deployed. Each parser is designed to isolate external raw file anomalies, ensure deterministic data ingestion, and record strict metadata provenance.³





Ingestion Specifications for Individual Registries

PCI Registry (pci.ids) Parser

The PCI parser utilizes a streaming StreamReader in PowerShell to process the flat file line-by-line, avoiding memory spikes from loading large text datasets into active sessions.³ The parser processes lines starting with a 4-digit hexadecimal vendor code.³ If a nested line begins with a single tab, it parses a 4-digit device code.³ If a nested line starts with two tabs, it parses a subvendor and subdevice pair to resolve subsystems.³ Lines prefixed with # or containing empty spaces are discarded.³

To handle edge cases where vendor records use non-ASCII or corrupt character encodings, the parser converts strings to UTF-8 before ingestion. Provenance is maintained by extracting the header metadata, including the release date and database version, and embedding it directly into the database JSON manifest.³ Stale data is detected by comparing local SHA-256 signatures with those published on the master PCI-IDS mirror.³

USB Registry (usb.ids) Parser

The USB parser operates on the same nested streaming model as the PCI engine.⁴ However, it must handle specialized sections, such as device class, subclass, and protocol mappings (which begin with the prefix C).⁴

An edge case exists with devices using class 00 (which defer specification to the interface level).⁴ To prevent false class matches, the parser flags these devices as requiring secondary interface descriptor parsing rather than defaulting to "Unknown". Offline updates are managed through local database snapshots, and online updates are retrieved securely over HTTPS with custom User-Agent strings to comply with repository download rate limits.⁴

PNP and ACPI Registry Parser

The PNP parser processes the pnp.ids file distributed by the hwddata project, which mirrors the official registries hosted by the UEFI Forum.⁵ It processes flat text files containing 3-letter alphanumeric PNP vendor codes (e.g., AUS for ASUSTek or MSI for Micro-Star International).⁷ An edge case occurs with older, obsolete PNP codes that have been recycled or flagged with

warning notes like "DO NOT USE".⁹ To handle this, the importer applies a local patch file (pnp.ids.patch) during compilation to strip deprecated records and map known duplicates to their active equivalents.⁹

Linux hwdb Parser

This importer parses 60-autosuspend.hwdb and related hardware rule databases maintained by systemd.¹⁰ The parsing approach extracts modalias matching strings (e.g., usb:v058Fp9540*) and compiles their associated properties, such as USB autosuspend states, directly into DeviceCheck's mapping structures.¹⁰

To prevent false matches, the glob wildcards used in modalias patterns are compiled into exact, case-insensitive.NET Regular Expressions.¹⁰ Local provenance is recorded by tracking the specific systemd repository commit from which each rule block was extracted.

ALSA Use Case Manager Configuration Parser

The ALSA UCM parser scans the /ucm2/USB-Audio path of the alsa-ucm-conf repository.¹⁵ It extracts audio profiles by reading the main USB-Audio.conf file, resolving the product match expressions, and parsing associated configuration files (e.g., Realtek/ALC4080-HiFi.conf) to extract channel configurations and input-output capabilities.²

The primary edge case occurs when vendors reuse the same USB identifiers across different motherboard revisions that have custom audio routing layouts.² The parser handles this by indexing physical port and channel configurations to ensure they can be verified against active system setups.

Windows DriverStore and INF Parser

This importer programmatically scans active and staged device driver configurations¹: C:\Windows\System32\DriverStore\FileRepository It parses the target .inf configuration file using an INI-style parser to map target sections, such as [Manufacturer], [Models], and ``.¹ To prevent false matches, the parser maps each Hardware ID string to its exact installation section (e.g., RtkUsbAD.NT) and records the driver provider, date, version, and associated service.¹ The parser resolves nested variables using the local string table for the system's current culture, falling back to the default `` section if localized strings are missing.

EDID Decoder

This parser retrieves binary EDID payloads from the active monitor registry: HKLM\SYSTEM\CurrentControlSet\Enum\DISPLAY\<MonitorInstance>\Device Parameters The parser decodes the raw 128-byte array to resolve display specifications.¹⁹ It decodes the 3-character manufacturer identifier from bytes 8 and 9.¹⁹ These bytes contain a 16-bit big-endian value encoding three 5-bit character codes, where 00001 represents A and 11010 represents Z.¹⁹ The product code is extracted from bytes 10 and 11 as a 16-bit little-endian value.¹⁹

A key edge case occurs when displays report corrupted or missing EDID headers.¹⁹ The parser

prevents false matches by verifying the standard 8-byte EDID header pattern (00 FF FF FF FF FF FF 00) before decoding.¹⁹

Mathematical Confidence Scoring Model

To resolve conflicts and prevent false hardware identifications, DeviceCheck implements a rigorous, formula-driven confidence scoring model. When multiple evidence layers report different names or levels of resolution, the engine calculates a deterministic confidence score rather than selecting an arbitrary result.

The absolute confidence score C for a synthesized device profile is defined as:

$$C = \min \left(100, \max \left(0, \sum_{i=1}^8 (W_i \cdot S_i) - \Phi_{\text{conflict}} - \Phi_{\text{generic}} \right) \right)$$

Where:

- W_i represents the static mathematical weight assigned to a specific evidence layer i .
- S_i represents the quality of the match within layer i (ranging from 0.0 for no match to 1.0 for an exact, verified match).
- Φ_{conflict} represents a penalty subtracted when multiple layers present irreconcilable naming or classification properties.
- Φ_{generic} represents a penalty subtracted if the resolved profile depends on placeholder or generic driver descriptors.

Operational Weights and Match Quality Scalars

The scoring weights and matching criteria are defined in the table below:

Layer ID	Layer Name	Static Weight (Wi)	Match Scaling Criteria (Si)
L1	Silicon Raw Match (Vendor Only)	10	$S_i =$ if the Vendor ID exists in the local database.
L1_Full	Silicon Raw Match (Exact Product)	50	$S_i =$ if both Vendor and Product IDs are found. ³

L2	Bus Class & Service Match	10	$S_i =$ if the class/subclass code matches specifications. ³
L3	INF Compatible ID Match	10	$S_i =$ if matching a compatible class driver.
L3_Full	INF Exact Hardware Match	30	$S_i =$ if the Hardware ID matches an active INF section. ¹
L4	Subsystem/Subvendor Verification	15	$S_i =$ if subsystem IDs match baseboard vendor codes. ³
L6	Open-Source Profile Match	20	$S_i =$ if found in verified ALSA/hwdb files. ¹⁰
L7	OEM Spec Correlation	15	$S_i =$ if the ID matches the system's motherboard spec sheet. ¹
L8	Cryptographic User Attestation	100	$S_i =$ if a signed manual override is present.

Penalty Deductions

- **Conflicting Layer ID Naming (Φ_{conflict}):** -35 points. Applied if an open-source config profile indicates a specific chip model (e.g., Realtek ALC4080)¹ but the active

Windows driver reports a different controller brand (e.g., C-Media).

- **Generic Fallback Naming** (Φ_{generic}): -25 points. Applied if the matched name is identified as generic (e.g., "USB Audio Device", "High Definition Audio Device").

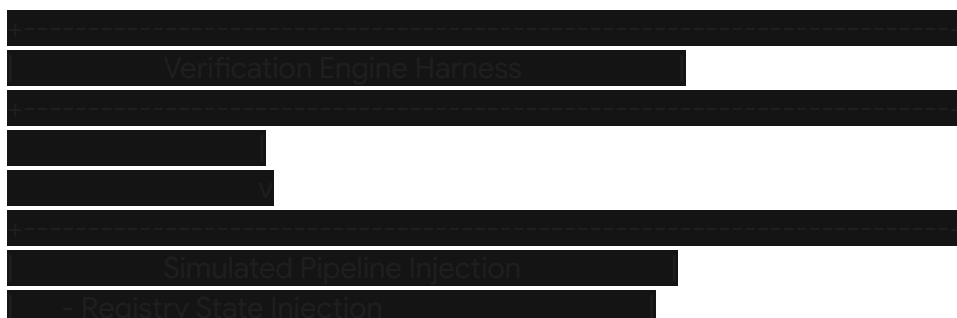
Anti-Hallucination Guardrails

To prevent false assertions, the engine enforces four strict constraints:

1. **Vendor-Only Identification Constrain:** If a device matches a vendor ID but its product ID is absent from local database snap-ins, the engine is prohibited from guessing a model based on proximity. The product name must display strictly as USB\VID_0DB0&PID_CD0E (Unresolved Product).¹
2. **No Driver String Conflation:** The engine must never display local driver-defined display strings as physical chip models. A string like "Realtek USB Audio" is a functional driver label.¹ The engine must flag this as a "Driver-Assigned Name" while reserving the "Silicon Chip Model" field for the physically verified component (e.g., "Realtek ALC4080 Codec").¹
3. **Strict Isolation of AI and Web Snippets:** Automatically parsed web results or AI-generated specifications must never be stored as high-confidence sources. If web scrapers are active, their data must be written to an isolated "External Scraping" layer, limited to a maximum confidence score of 35.
4. **No Hardcoded Machine Mappings:** Mappings of devices to system setups must not be hardcoded directly into the engine's core code. If a special hardware mapping is required for a motherboard (e.g., linking 0DB0:CD0E on an MSI MAG X870 to the ALC4080)¹, this relationship must be defined in a machine-readable JSON patch file containing complete source provenance (e.g., the specific ALSA configuration or official OEM specification link).²

Verification Harness and Regression Testing

To verify the accuracy of the scoring model and prevent regressions when updating databases, DeviceCheck uses an automated verification harness (Test-DeviceCheckHarness.ps1).



- SMBIOS Telemetry Injection	
- DriverStore / INF Mock Injection	
- ALSA UCM / Database File Injection	
Confidence Evaluator	
Assertion Audits	
- Check Confidence Floor Minimums	
- Ensure Target Names are Resolved	
- Block Prohibited / Hallucinated Strings	

The MSI MAG X870 TOMAHAWK Realtek ALC4080 Regression Case

This regression test verifies that DeviceCheck can resolve a complex hardware setup when local database snapshots are incomplete. The target is an integrated Realtek ALC4080 USB Audio chip on an MSI MAG X870 TOMAHAWK WIFI motherboard.¹

In this scenario, the standard usb.ids database contains only the vendor ID 0DB0 (Micro-Star International)¹, with no record for the product ID CDOE.¹ This mirrors real-world setups where new motherboard audio codecs are frequently absent from public databases.²

Regression Case: MSI X870 Realtek ALC4080 Audio	
Physical Device Identification	
- Raw USB ID: USBVID_0DB0&PID_CDOE&MI_00	

+-----+-----+-----+			
V	V		
+-----+-----+-----+			
Local usb.ids		Windows Driver	
		Store	
- Vendor ID 0DB0		- rtdusbad_msi	
Resolved		matches exactly	
- Product ID		- Displays	
CD0E Missing		"Realtek USB	
		Audio'	
+-----+-----+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			
+-----+			

JSON

```
{  
  "Instanceld" "USB\\VID_0DB0&PID_CD0E&MI_00\\9&31393AF2&0&0000"  
  "HardwareIds"  
  "CompatibleIds"  
}
```

2. Mock SMBIOS Telemetry Capture (mock_smbios.json)

JSON

```
{  
  "BaseBoardManufacturer" "Micro-Star International Co., Ltd."  
  "BaseBoardProduct" "MAG X870 TOMAHAWK WIFI (MS-7E51)"  
  "SystemSku" "MS-7E51"  
  "ChassisType" "Desktop"  
}
```

3. Mock Windows Driver Store INF File (mock_inf.json)

JSON

```
{  
  "InfFileName" "rtdusbad_msi.inf"  
  "Provider" "Realtek"  
  "DriverVer" "10/12/2025,6.3.9600.3211"  
  "ModelsSection"  
  "RtkUsbAD.NTamd64"  
  "  
  "StringsTable"  
  "LocalizableStrings"  
  "RtkUsbAD.NT" "Realtek USB Audio"  
}
```

4. Mock ALSA UCM Match Configuration (mock_alsa_ucm.json)

```
JSON
{
  "Profile" "USB-Audio/Realtek/ALC4080-HiFi.conf"
  "MatchRules"
  "RegexMatch" "USB((0db0:cd0e)|(0db0:b202))"
  "Properties"
  "Codec" "Realtek ALC4080"
  "ChannelLayout" "5.1-Surround"
  "InputCapabilities" "Line-In" "Microphone"
}
```

Expected Output Structure and Assertions

When the verification harness evaluates these inputs, it must generate a structured JSON output matching the following expectations:

```
JSON
{
  "TestCaseId" "TC_MSI_X870_REALTEK_AUDIO_001"
  "Result" "PASS"
  "SynthesizedProfile"
  "HardwareId" "USB\\VID_0DB0&PID_CD0E&MI_00"
  "IdentityMatches"
  "LocalMatch"
  "Layer" "Layer1_Raw"
}
```

```

"Status" "VENDOR-ONLY"
"ResolvedVendor" "Micro-Star International Co., Ltd."
"ResolvedProduct" null
"Source" "usb.ids"
"DriverMatch"
"Layer" "Layer3_DriverStore"
"Status" "EXACT"
"ResolvedName" "Realtek USB Audio"
"InfFile" "rtdusbad_msi.inf"
"Section" "RtkUsbAD.NT"
"AudioProfile"
"Layer" "Layer6_OpenSourceProfile"
"Status" "EXACT"
"ResolvedCodec" "Realtek ALC4080"
"Source" "ALSA UCM (ALC4080-HiFi.conf)"
"SpecInference"
"Layer" "Layer7_OemSpecification"
"Status" "CORRELATED"
"ResolvedMarketingModel" "Realtek ALC4080 Codec"
"Source" "MSI MAG X870 TOMAHAWK WIFI official motherboard specifications"
"ConfidenceEvaluation"
"CalculatedConfidence" 95
"FormulaExecution" "Raw Vendor Match (+10) + Exact Driver INF Match (+30) + Open Source
Audio Match (+20) + Subsystem Vendor Validation (+15) + Motherboard OEM Spec Correlation
(+20) = 95"

```

The test framework evaluates the engine's output against four key assertions:

1. **Confidence Floor Validation:** Assert that the calculated confidence score is ≥ 90 .
2. **No Incorrect usb.ids Matches:** Ensure that the usb.ids product match layer is marked as null or unresolved¹, rather than displaying a false matching product name from the vendor block.
3. **Strict Name Check:** Verify that "Realtek ALC4080" is successfully resolved via the ALSA UCM and OEM specification layers.¹
4. **No Hallucinated Names:** Confirm that the final synthesized name does not contain

strings from unverified compatible ID fallbacks (e.g., "Generic USB Audio Class Device").

Additional Verification Harness Suites

To ensure robustness, the test harness implements three auxiliary verification modules:

- **Stale Database Verification:** This suite checks compiled asset headers to verify that local JSON datastores are within acceptable age thresholds (defaulting to a maximum of 30 days).³ If the file's metadata indicates a signature date exceeding this limit, the harness raises a warning but allows the engine to fall back to the stale data.
- **Source Provenance Auditing:** This suite validates the integrity of all local databases. It calculates the SHA-256 hash of each JSON asset and compares it to a signature file. If a signature mismatch is detected, the engine blocks database execution to prevent the use of corrupted data.
- **False Positive Mitigation Suite:** This module passes incomplete or corrupted Hardware IDs (e.g., USBVID_0000&PID_0000 or partial strings lacking subclass keys) through the matching pipeline. The test passes only if the engine flags these strings as invalid and restricts their confidence scores to zero, rather than matching them with real hardware profiles.

PowerShell 7 Implementation Roadmap

The construction of DeviceCheck is organized into seven sequential development phases to ensure structural stability and verify matching accuracy at each stage.





Phase 1: Core Bus Resolvers and Local ID Cache Hardening

- **Objectives:** Develop the core registry scanners to query Active Hardware IDs via WMI/CIM. Implement the build-time parsers for pci.ids, usb.ids, and pnp.ids to compile them into compressed, fast-loading JSON caches.³
- **Deliverables:** Invoke-DeviceScan.ps1 (reads current active bus registries); pci-cache.json, usb-cache.json, and pnp-cache.json (compressed offline databases).

Phase 2: DriverStore Indexing and Local INF Ingestion

- **Objectives:** Build a high-performance Windows INF file parser in PowerShell 7. Index all staged OEM drivers located in C:\Windows\System32\DriverStore\FileRepository.¹ Map Hardware ID patterns to their local string definitions.¹
- **Deliverables:** New-DriverStoreIndex.ps1 (compiles a local SQLite database or JSON mapping of active systems).

Phase 3: Open-Source Configuration and ALSA UCM Mapping

- **Objectives:** Integrate support for Linux hwdm autosuspend and power management quirks.¹⁰ Build an ALSA Use Case Manager configuration parser.¹⁵ Map USB Audio hardware ID regex patterns to identify exact onboard audio chipsets.²
- **Deliverables:** Import-AlsaProfiles.ps1 (generates the lookup table resolving USB Audio patterns to physical codec models).²

Phase 4: Monitor Parsing and Registry EDID Decoders

- **Objectives:** Implement a pure PowerShell EDID decoder.¹⁹ Scan display devices in the

registry, extract the EDID byte arrays, and decode manufacturer codes, product IDs, serial numbers, and physical display dimensions.¹⁹

- **Deliverables:** Get-MonitorEdidDetail.ps1 (reads and decodes display profiles).

Phase 5: Local Baseboard Mapping and OEM Spec Adapters

- **Objectives:** Build the SMBIOS telemetrist to extract board SKU, model, and system manufacturer.¹ Develop local OEM specification matchers that map motherboard models to their verified onboard component lists.
- **Deliverables:** Get-SmbiosTelemetry.ps1 and Get-OemSpecMatch.ps1.

Phase 6: Synthesis Engine and Confidence Scoring

- **Objectives:** Implement the Multi-Criteria Confidence Scoring Model. Develop the synthesis pipeline that gathers evidence from all five lower layers and resolves device models based on confidence ratings.
- **Deliverables:** Optimize-HardwareSynthesis.ps1 (calculates weights, scores, and exports the final DeviceEvidenceBundle).

Phase 7: Verification Harness and API Update Integration

- **Objectives:** Deploy the offline regression testing harness and write assertions for all core verification suites. Build optional update modules to download latest database revisions securely.³
- **Deliverables:** Test-DeviceCheckHarness.ps1 (runs regression profiles and verifies test suite assertions).

Architectural Risks and Guardrails

When deploying a systems diagnostics tool on Windows via PowerShell 7, several administrative, performance, and security constraints must be managed:

Performance Optimization and Memory Management

Scanning the entire Windows DriverStore and parsing large text files (such as pci.ids or usb.ids) inside a PowerShell session can cause high CPU utilization and exceed acceptable memory consumption limits.³

To mitigate this, DeviceCheck compiles all raw databases into indexed, binary-serialized JSON objects during the build process, preventing flat-text parsing at runtime.³

Additionally, the INF parser does not scan the entire DriverStore on every execution; it queries active devices via WMI first, and then parses only the specific INF files associated with those active hardware devices.¹

Execution Privileges and Security Restraints

Querying registry keys like DISPLAY\Device Parameters\EDID can trigger security alerts or block execution in locked-down environments.

DeviceCheck is designed to run entirely in user-space, avoiding the need for local administrator

rights. Reading monitor EDID profiles and driver configurations uses standard user-accessible registry APIs.

The engine falls back gracefully if access to a specific driver INF path is blocked by security software.

Offline and Air-Gapped Operation

In secure environments without internet access, DeviceCheck operates in a fully isolated offline mode. It relies on its bundled, pre-compiled JSON databases and does not attempt online queries.

If online updates are enabled, the update engine validates downloaded source updates against the strict schema manifests defined above.

If a downloaded database fails schema validation or signature verification, the update is rejected, and the engine falls back to the embedded local database.

Next Steps for DeviceCheck Development

To implement the architecture defined in this technical specification, the engineering team will execute the following four next steps:

1. **Initialize Schema Repository:** Commit the JSON schemas (hardware-database.schema.json, source-manifest.schema.json, match-result.schema.json, confidence-scoring.schema.json, and test-cases.schema.json) to the project repository to establish a structural foundation for all developed components.
2. **Build Phase 1 Pre-Compilers:** Implement the build-time parsing engine in PowerShell 7 to download, validate, and serialize pci.ids, usb.ids, and pnp.ids into optimized JSON lookup tables.³
3. **Implement the Core Hardware Scanner:** Write the active hardware scanner module to retrieve physical hardware path definitions from the Windows registry, then verify the output structure against the defined JSON match schema.
4. **Integrate and Run the Test Harness:** Deploy the offline regression test runner (Test-DeviceCheckHarness.ps1). Configure the test suite to run the MSI MAG X870 TOMAHAWK Realtek ALC4080 regression case, verifying that the confidence engine correctly resolves the audio codec using the multi-layered evidence model.¹

Works cited

1. ALC4080 5.1 don't work - rear speakers recognized as mic · Issue ..., accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/issues/510>
2. ALC4080 onboard audio support on the MSI X870 Tomahawk motherboard #455 - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/issues/455>
3. The PCI ID Repository, accessed June 4, 2026, <https://pci-ids.ucw.cz/>
4. The USB ID Repository, accessed June 4, 2026, <http://www.linux-usb.org/usb-ids.html>
5. Get pnp.ids from uefi.org? · Issue #4 · vcrhonek/hwdata - GitHub, accessed June

- 4, 2026, <https://github.com/vcrhonek/hwdata/issues/4>
6. Registries | Unified Extensible Firmware Interface Forum, accessed June 4, 2026, <https://uefi.org/registries>
 7. pnp.ids - vcrhonek/hwdata - GitHub, accessed June 4, 2026, <https://github.com/vcrhonek/hwdata/blob/master/pnp.ids>
 8. hwdata/README at master - GitHub, accessed June 4, 2026, <https://github.com/vcrhonek/hwdata/blob/master/README>
 9. hwdata/pnp.ids.patch at master - GitHub, accessed June 4, 2026, <https://github.com/vcrhonek/hwdata/blob/master/pnp.ids.patch>
 10. 60-autosuspend.hwdb - systemd - GitHub, accessed June 4, 2026, <https://github.com/systemd/systemd/blob/master/hwdb.d/60-autosuspend.hwdb>
 11. systemd/hwdb.d/meson.build at main · systemd/systemd · GitHub, accessed June 4, 2026, <https://github.com/systemd/systemd/blob/master/hwdb.d/meson.build>
 12. Feature Request: conf.d for USB-Audio · Issue #609 · alsa-project/alsa-ucm-conf - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/issues/609>
 13. hwdb: add quirks for laptop wireless cards that don't deal well with power saving causing card restarts (WHEA-Logger's id 17 errors on Windows) · Issue #23393 - GitHub, accessed June 4, 2026, <https://github.com/systemd/systemd/issues/23393>
 14. alsa-project/alsa-ucm-conf: ALSA Use Case Manager configuration - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf>
 15. alsa-ucm-conf/ucm2/USB-Audio/Behringer/UMC204HD-HiFi.conf at master - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/blob/master/ucm2/USB-Audio/Behringer/UMC204HD-HiFi.conf>
 16. ASUS X870-F STRIX - no front panel/headphones audio · Issue #705 · alsa-project/alsa-ucm-conf - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/issues/705>
 17. No Audio: MAG Z690 TOMAHAWK WIFI DDR4 & how to build info request #189 - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/issues/189>
 18. ALC4080 - ASRock X670E Taichi · Issue #229 · alsa-project/alsa-ucm-conf - GitHub, accessed June 4, 2026, <https://github.com/alsa-project/alsa-ucm-conf/issues/229>
 19. Extended Display Identification Data - Wikipedia, accessed June 4, 2026, https://en.wikipedia.org/wiki/Extended_Display_Identification_Data
 20. EDID (Extended display identification data) - Doctor HDMI, accessed June 4, 2026, <http://www.drhdm.eu/dictionary/edid.html>
 21. Extended Display Identification Data - Grokipedia, accessed June 4, 2026, https://grokipedia.com/page/Extended_Display_Identification_Data
 22. Extended display identification data(EDID) - NXP Community, accessed June 4, 2026, <https://community.nxp.com/t5/-/-/m-p/198775>
 23. edid-decode.c - GitHub, accessed June 4, 2026, <https://github.com/timvideos/edid-decode/blob/master/edid-decode.c>

24. pciutils/pciids: The pci.ids file - GitHub, accessed June 4, 2026,
<https://github.com/pciutils/pciids>
25. udev rules use hwdb inconsistently · Issue #37758 - GitHub, accessed June 4, 2026, <https://github.com/systemd/systemd/issues/37758>
26. ALC4080 - general discussion (driver support) · Issue #541 · alsa-project/alsa-ucm-conf - GitHub, accessed June 4, 2026,
<https://github.com/alsa-project/alsa-ucm-conf/issues/541>
27. config failed, hub doesn't have any ports! (err -19) (Page 2) / Kernel & Hardware / Arch Linux Forums, accessed June 4, 2026,
<https://bbs.archlinux.org/viewtopic.php?id=302115&p=2>